AI Forward Deployed Engineer

The AI Architect & FDE Course



3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

- AI Solutions Architect
- Security, Guardrails, API Gateways
- Enterprise Context Layer & Integrations
- Production Deployment
- Capstone

Week 2: Become an AI Architect

- Architecture, Evals, Observability
- Agentic RAG, MCP, Memory
- Durable State & Human in the Loop
- Agent Teams, Swarms, Fleet
- Self-Improvement, RSI

Week 1: Become an AI Engineer — AI ENGINEER

- Path to FDE
- Frontier LLMs & Agents
- Coding Agents & ADLC
- Open-Source, Local, Token Cost
- Harness Engineering

Taking stock

You can already:

- [x] Describe how an FDE builds AI solutions to solve business problems
- [x] Use evaluations and observability in Langfuse to drive business outcomes
- [x] Build agents that have memory with Agentic RAG over MCP
- [x] Create durable and scalable agents in LangGraph with a human in the loop

By the end of today, you'll be able to:

- [ ] Explain recursive self-improvement and how it applies to agents
- [ ] Use an LLM as a judge with a reference answer
- [ ] Build an agent that improves another agent at a business task
- [ ] Add authentication to an AI app with Supabase Auth

Recap

We ran a swarm of claim agents and scored every run



Teams, swarms and fleets

Many agents working in parallel



A graph with no LLM

Send gives each claim its own agent



Limits, caps and retries

max_concurrency and RetryPolicy



The scoreboard

Quality, cost and time for every run

Self-improvement

On Day 1 we improved our agent by hand



Read the results
The scores and the claims it got wrong

Propose a change
Prompt, model or reasoning level

Approve it
A human checks it

Evaluate again
On the evaluation and held-out sets

Keep the change or roll it back

Today a new agent called the improver runs this loop.. and the improver can improve itself

LLM as a judge is a popular way to evaluate

We will use a particular approach: judge against a reference answer

- **A** — Same grounds and section, clearly better expressed
- **B** — Same grounds and same section
- **C** — Partly the same, with a ground or section missing
- **F** — Different grounds, or contradicts the handler

Jev as the judge

The decision model from Week 1 Day 2 grades each explanation

```
GRADE = {
   "type": "score",
   "instructions": "How well does `agent` match `handler`?",
   "criteria": ["F: different grounds", "C: partly the same",
                "B: same grounds and section",
                "A: same, and better expressed"],
}
response = openrouter_decisions.alpha.decisions.create(
   model="typesafe/jev-1.13",
   state={"handler": handler, "agent": agent},
   questions={"grade": GRADE},
)
grade = response.answers["grade"].score  # 0 (F) to 3 (A)
```



Fast
0.3 seconds a claim



Cheap
$0.00003 a claim

What the improver optimizes

A better version that never falls below the business objectives

Improve



Accuracy



The judge's grade



Cost per claim



Time per claim

Never below the objectives



Accuracy on the held-out set



A floor on the judge's grade



The spend budget

A change is kept when one measure improves beyond the noise and none gets much worse

The improver and its recipe

A new agent with a recipe as its instructions



Read the results
The latest run and its mistakes

Propose a change
Prompt, model or reasoning level

A human approves
Seen as a diff

Run the evaluation
Both sets, by the dispatcher

Improve the recipe
From this round and earlier ones

The improver's tools

`read_results` `propose_change` `run_evaluation` `propose_recipe`

Setting up Supabase Auth

Handlers sign in before they sign off or start a dispatch

In Supabase

Handlers
Added by hand with one password

Email
Auto-confirmed, so nothing is sent

Sign-ups
Turned off

In your .env file

Supabase URL and publishable key
Public and sent to the browser

Handler password
For Claude Code's browser checks

No Supabase secret
The backend checks sign-ins with Supabase's public key

The lab ends with this setup and a credit limit on the OpenRouter key

Today’s lab

In labs/week2day5.ipynb and its free twin



A judge with a reference

Jev grades each rationale against the handler’s reason

Version 1 and its scores

The baseline on the evaluation and held-out sets

One round of improvement

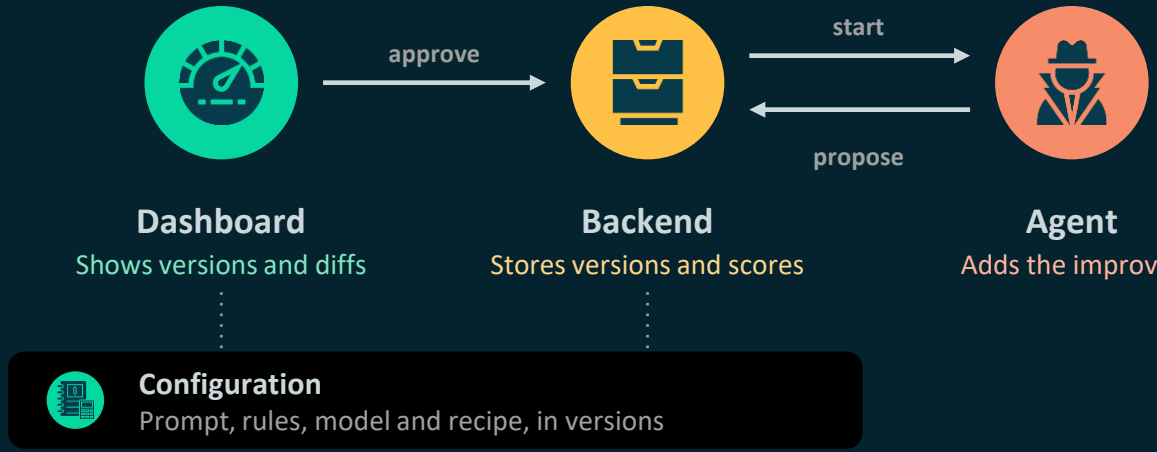
The improver proposes, you approve, code keeps or rolls back

Improving the improver

The recipe rewrites itself for round two

Upgrading the product

Every change the improver proposes is approved on the dashboard



Also in this build: Jev as the judge, a spend budget and a sign-in

Building it with Claude Code

The change is written up in labs/week2day5_product_IMPROVE.md

1. **Copy the brief into docs**
   cp labs/week2day5_product_IMPROVE.md docs/IMPROVE.md
2. **Ask Claude Code for a plan**
   Please plan self-improvement in docs/IMPROVE.md
3. **Let it build and commit each phase**
   Please implement the plan for self-improvement
4. **Sign in and run a round**
   Use a handler you added at the end of the lab
5. **Or check out my version**
   git checkout week2day5-start or git checkout week2day5-end

End of week assignments

Two challenges to wrap up the week



Run more rounds

Keep self-improving and recursively self-improving to find the fastest, cheapest solution



Solve a problem of your own

Follow all the steps of this week to apply a similar solution to a personal or professional pain point, using LangGraph for durable, scalable workflows and a human in the loop

Everything you can do

Coming right up

Week 1

AI Engineering

Agent loops and tools
Frontier LLMs and agents
Model selection
Agent frameworks
Coding agents
The ADLC
Open-source & local models
Token cost
Context engineering
Harness engineering

Technologies

OpenAI SDK
OpenRouter
LangChain
Claude Code
Claude Agent SDK

Week 2

AI Architecture

System design
Agent architecture
Evals and observability
Cost and speed trade-offs
Agentic RAG and memory
MCP
Durable state
Human in the loop
Agent swarms
Self-improvement

Technologies

Langfuse
Qdrant / vector db
MCP
LangGraph
Supabase

Week 3

Forward Deployed Engineering

Customer discovery
Governance & compliance
Security and guardrails
API and MCP gateways
Enterprise context layer
Enterprise integrations
Cloud infrastructure
Production deployment
Operational handover
Adoption

Technologies

Serverless
AWS AgentCore Gateway
AWS AgentCore Policy
AWS AgentCore Memory
AWS AgentCore Runtime

3 weeks to Forward Deployed Engineer

Theory Practical Projects



3 weeks to Forward Deployed Engineer

Theory Practical Projects

Week 3: Become an AI FDE

AI Solutions Architect	Security, Guardrails, API Gateways	Enterprise Context Layer & Integrations	Production Deployment	Capstone

Week 2: Become an AI Architect — AI ARCHITECT

Architecture, Evals, Observability	Agentic RAG, MCP, Memory	Durable State & Human in the Loop	Agent Teams, Swarms, Fleet	Self-Improvement, RSI

Week 1: Become an AI Engineer — AI ENGINEER

Path to FDE	Frontier LLMs & Agents	Coding Agents & ADLC	Open-Source, Local, Token Cost	Harness Engineering

AI Forward Deployed Engineer

The AI Architect & FDE Course

